ECB, CBC and CFB Modes

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

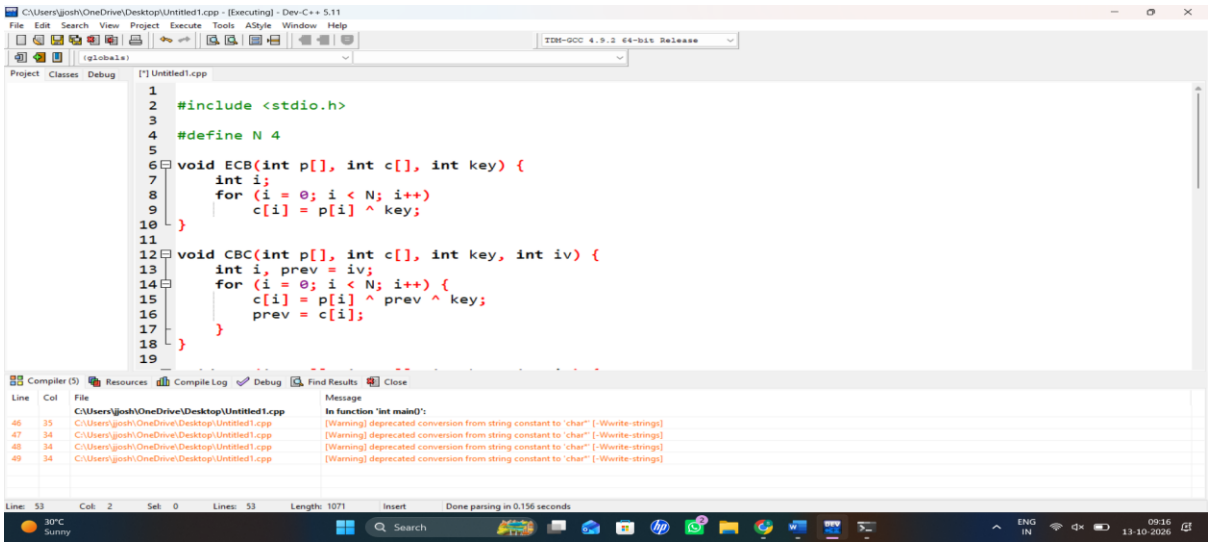
Aim

To write a C program to demonstrate ECB, CBC, and CFB modes of operation.

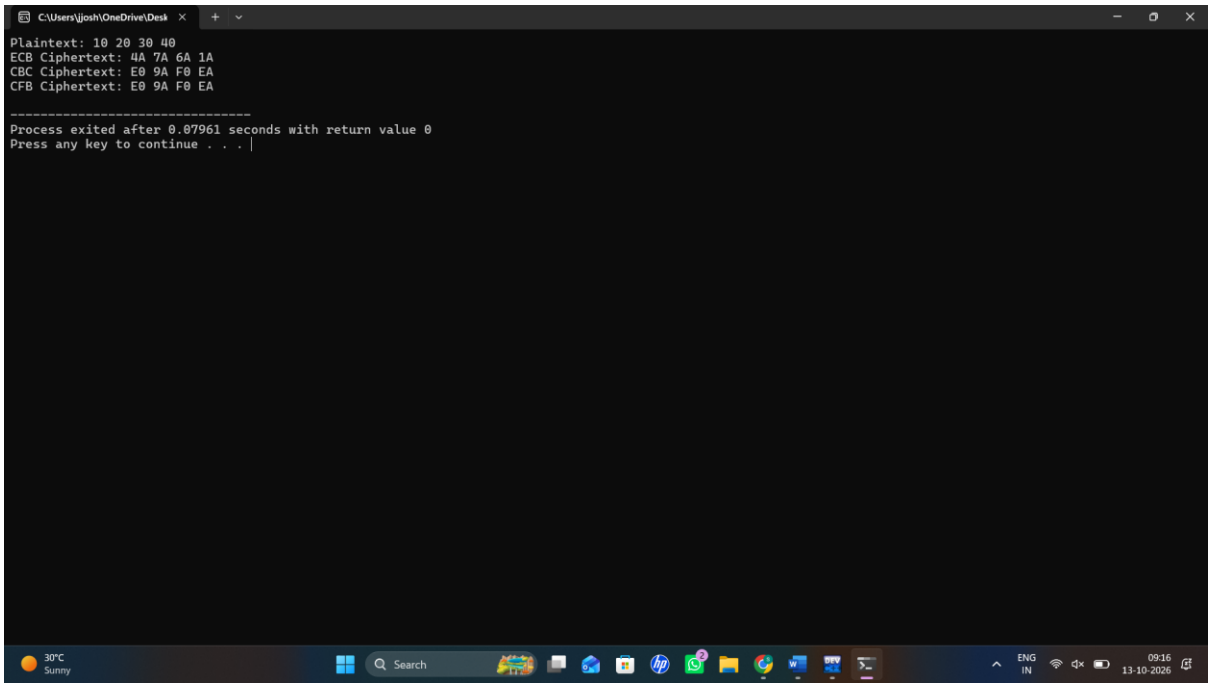
Algorithm

1. Start the program.
2. Initialize the plaintext, encryption key, and initialization vector (IV).
3. Encrypt each plaintext block independently using ECB mode.
4. Encrypt plaintext blocks using CBC mode with XOR and feedback.
5. Encrypt plaintext blocks using CFB mode with feedback.
6. Display the ciphertext for all three modes.
7. Stop the program.

Program :



Output:



Result

Thus, a C program demonstrating ECB, CBC, and CFB modes was implemented successfully.

Motivation for padding every message

Padding every message, even when the last block is already complete, helps the receiver distinguish the actual end of the message from padding. In particular, using a padding rule such as a 1 bit followed by zero bits allows the receiver to identify and remove padding unambiguously, provided the original message length is not otherwise known. It also helps avoid ambiguity between a message that naturally ends in zero bits and one that has been padded.